

Auth and Cloudflare Handling – Load Test Reference

Project: Pharmacy Patient Portal – Consent Load Testing
Environment: DEV – `pharmacy-patient-portal.dev.apps.lilly.com`
Date: 2026-08-10

1. Authentication Flow (Microsoft SSO)

1.1 How Auth Works

The application uses Microsoft Azure AD (AAD) SSO with an OAuth2 proxy layer. The session is maintained via a `_oauth2_proxy` cookie on the application domain.

1.2 Bootstrap Process

Step	What Happens
1	Run <code>npm run auth:bootstrap</code> from project root
2	Real Chromium browser opens and navigates to the app URL
3	You complete Microsoft SSO login manually (including MFA if required)
4	Playwright waits for the post-login redirect back to the app
5	All session cookies are saved to <code>playwright-load/auth-state-ms.json</code>

1.3 Session Reuse in Tests

Tool	How Auth State Is Used
Playwright	<code>browser.newContext({ storageState: AUTH_STATE_PATH })</code> — context starts pre-authenticated

k6

open(AUTH_STATE_PATH) at init time, context.addCookies(AUTH_COOKIES)

browser

before navigation

1.4 Key Auth Cookies

Cookie Name	Domain	Purpose
_oauth2_proxy	pharmacy-patient-portal.dev.apps.lilly.com	Main app session — required for all authenticated page loads
ESTSAUTHPERSISTENT	.login.microsoftonline.com	Microsoft persistent SSO token
ESTSAUTH	.login.microsoftonline.com	Microsoft session token
esctx-*	.login.microsoftonline.com	Extended SSO context cookies

1.5 Session Expiry

Scenario	Symptom	Fix
AAD token expired	Tests return 403 or redirect to login.microsoftonline.com	Re-run <code>npm run auth:bootstrap</code>
k6 shows no SSO redirect check failing	All VUs fail at first page load	Re-run <code>npm run auth:bootstrap</code>
Playwright shows not-authenticated in sampleFailures	Auth state stale	Re-run <code>npm run auth:bootstrap</code>

Note: Microsoft AAD session tokens typically last 8–24 hours depending on tenant policy. Always re-run `npm run auth:bootstrap` at the start of each test day.

2. Cloudflare Bot Protection Bypass

2.1 How Cloudflare Is Handled

Cloudflare bot protection is bypassed automatically because both Playwright and k6 browser use **real Chromium instances**, not HTTP API calls.

Cloudflare Check	How It Is Bypassed
JavaScript challenge	Real browser executes the challenge automatically
Browser fingerprint	Real Chromium matches expected browser fingerprint
TLS fingerprint (JA3)	Real Chromium TLS matches expected client hello signature
Cookie validation (<code>__cf_bm</code>)	Captured during <code>auth:bootstrap</code> and injected with auth cookies
Mouse/interaction patterns	Not checked for page loads; only checked for form submissions

2.2 Cloudflare Cookies in Auth State

Cookie Name	Domain	Rotation Interval
<code>__cf_bm</code>	<code>.dev.apps.lilly.com</code>	Every 30 minutes
<code>__cf_bm</code>	<code>.dev.fdl.lilly.com</code>	Every 30 minutes
<code>__cf_bm</code>	<code>.dev.unex.lilly.com</code>	Every 30 minutes

Important: The `__cf_bm` cookie rotates every 30 minutes. VUs that started before rotation continue to work. New VUs starting after rotation may get challenged. For long-running tests (>30 minutes), some VUs may see Cloudflare challenges — real browsers handle these automatically.

3. Consent Banner Handling

3.1 Why Special Handling Is Required

The Transcend consent manager uses a **closed Shadow DOM** (`mode: "closed"`). This means:

- `element.shadowRoot` returns `null` from external JavaScript
- Standard CSS selectors cannot pierce into the shadow DOM
- k6 browser cannot click buttons inside it via normal DOM interaction

3.2 Solution Per Tool

Tool	Approach	How It Works
Playwright	Shadow DOM wait + evaluate	<code>waitForSelector</code> waits for host to attach, then clears consent cookies, reloads, and clicks Accept/Reject inside shadow DOM via <code>page.evaluate()</code>
k6 browser	Direct storage write	Clears existing <code>tcm</code> cookie and <code>tcmConsent</code> <code>localStorage</code> , writes the correct consent payload directly, then verifies both are readable — bypasses shadow DOM entirely

3.3 Consent Payload Written by k6

Accept All:

```
{
  "purposes": {
    "Functional": true, "Analytics": true, "Advertising": true,
    "ProgramTcpaTpo": "Auto", "EnterpriseHipaaAuthorization": "Auto",
    "EnterpriseMarketing": "Auto", "EnterpriseMarketResearch": "Auto",
    "EnterpriseMedicalResearch": "Auto", "EmailCommunicationPreference": "Auto",
    "ProgramTcpaMarketing": "Auto", "SaleOfInfo": "Auto"
  },
  "confirmed": true, "prompted": true, "updated": true
}
```

Reject All:

```
{ same structure but "Analytics": false, "Advertising": false }
```

4. Concurrency Limits and Scaling

Users	Tool	RAM Required	Status
1–20	Playwright	~3 GB	Confirmed working

1–20	k6 browser	~3 GB	Confirmed working
50	k6 browser	~7–8 GB	Requires 16 GB+ RAM machine
100	k6 browser	~15 GB	Not feasible on single machine
200+	k6 HTTP	Very low	Works (no browser, no DOM verification)
200+	Distributed k6	Distributed across machines	Requires multiple machines or CI agents

5. Troubleshooting Quick Reference

Error	Cause	Fix
not-authenticated in Playwright failures	Auth state expired	<code>npm run auth:bootstrap</code>
no SSO redirect failing in k6	Auth cookies expired or path wrong	<code>npm run auth:bootstrap</code>
host-not-found in k6	Shadow DOM closed — fixed by direct storage write	Current k6 script handles this automatically
Access is denied CrashpadMetrics	Windows file lock on Chromium temp dir — harmless	Add <code>2>nul</code> to CMD command
process unexpectedly ended exit status 1	Corporate security or <code>--disable-gpu</code> issue	Removed <code>--disable-gpu</code> from k6 browser args
403 on page load	Auth state expired or WAF rate limit	Re-run bootstrap; reduce concurrency if WAF
interrupted iterations at test end	Normal — VUs mid-iteration when test	Not an error; check <code>consent_success_rate</code>

duration ended

threshold

To export as PDF: Open this file in Chrome or Edge → File → Print → Save as PDF